11

Usability

Any darn fool can make something complex; it takes a genius to make something simple.
—Albert Einstein

Usability is concerned with how easy it is for the user to accomplish a desired task and the kind of user support the system provides. Over the years, a focus on usability has shown itself to be one of the cheapest and easiest ways to improve a system's quality (or more precisely, the user's perception of quality).

Usability comprises the following areas:

- *Learning system features*. If the user is unfamiliar with a particular system or a particular aspect of it, what can the system do to make the task of learning easier? This might include providing help features.
- *Using a system efficiently*. What can the system do to make the user more efficient in its operation? This might include the ability for the user to redirect the system after issuing a command. For example, the user may wish to suspend one task, perform several operations, and then resume that task.
- *Minimizing the impact of errors*. What can the system do so that a user error has minimal impact? For example, the user may wish to cancel a command issued incorrectly.
- *Adapting the system to user needs.* How can the user (or the system itself) adapt to make the user's task easier? For example, the system may automatically fill in URLs based on a user's past entries.
- *Increasing confidence and satisfaction*. What does the system do to give the user confidence that the correct action is being taken? For example, providing feedback that indicates that the system is performing a long-running task and the extent to which the task is completed will increase the user's confidence in the system.

11.1 Usability General Scenario

The portions of the usability general scenarios are these:

- *Source of stimulus.* The end user (who may be in a specialized role, such as a system or network administrator) is always the source of the stimulus for usability.
- *Stimulus*. The stimulus is that the end user wishes to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system.
- *Environment.* The user actions with which usability is concerned always occur at runtime or at system configuration time.
- *Artifact*. The artifact is the system or the specific portion of the system with which the user is interacting.
- *Response*. The system should either provide the user with the features needed or anticipate the user's needs.
- *Response measure.* The response is measured by task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs.

Table 11.1 enumerates the elements of the general scenario that characterize usability.

Figure 11.1 gives an example of a concrete usability scenario that you could generate using Table 11.1: The user downloads a new application and is using it productively after two minutes of experimentation.

TABLE 11.1 Usability General Scenario

Portion of Scenario	Possible Values
Source	End user, possibly in a specialized role
Stimulus	End user tries to use a system efficiently, learn to use the system, minimize the impact of errors, adapt the system, or configure the system.
Environment	Runtime or configuration time
Artifacts	System or the specific portion of the system with which the user is interacting
Response	The system should either provide the user with the features needed or anticipate the user's needs.
Response Measure	One or more of the following: task time, number of errors, number of tasks accomplished, user satisfaction, gain of user knowledge, ratio of successful operations to total operations, or amount of time or data lost when an error occurs

11.2 Tactics for Usability

Recall that usability is concerned with how easy it is for the user to accomplish a desired task, as well as the kind of support the system provides to the user. Researchers in human-computer interaction have used the terms *user initiative*, *system initiative*, and *mixed initiative* to describe which of the human-computer pair takes the initiative in performing certain actions and how the interaction proceeds. Usability scenarios can combine initiatives from both perspectives. For example, when canceling a command, the user issues a cancel—user initiative—and the system responds. During the cancel, however, the system may put up a progress indicator—system initiative. Thus, cancel may demonstrate mixed initiative. We use this distinction between user and system initiative to discuss the tactics that the architect uses to achieve the various scenarios.

Figure 11.2 shows the goal of the set of runtime usability tactics.

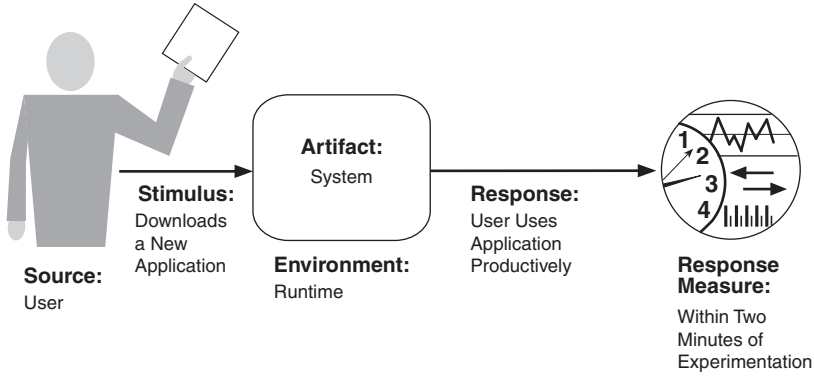

FIGURE 11.1 Sample concrete usability scenario

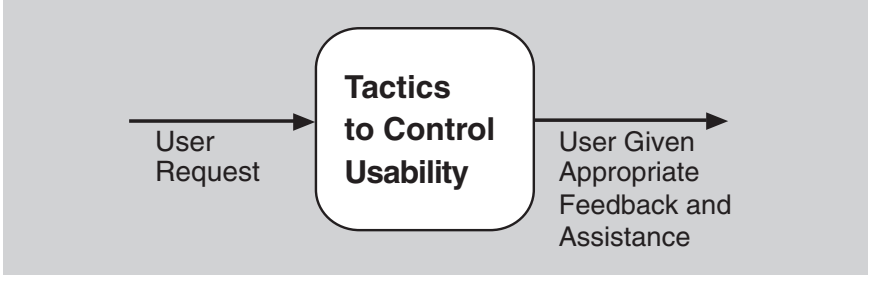

FIGURE 11.2 The goal of runtime usability tactics

Separate the User Interface!

One of the most helpful things an architect can do to make a system usable is to facilitate experimentation with the user interface via the construction of rapid prototypes. Building a prototype, or several prototypes, to let real users experience the interface and give their feedback pays enormous dividends. The best way to do this is to design the software so that the user interface can be quickly changed.

Tactics for modifiability that we saw in Chapter 7 support this goal perfectly well, especially these:

- Increase semantic coherence, encapsulate, and co-locate related responsibilities, which localize user interface responsibilities to a single place
- Restrict dependencies, which minimizes the ripple effect to other software when the user interface changes
- Defer binding, which lets you make critical user interface choices without having to recode

Defer binding is especially helpful here, because you can expect that your product's user interface will face pressure to change during testing and even after it goes to market.

User interface generation tools are consistent with these tactics; most produce a single module with an abstract interface to the rest of the software. Many provide the capability to change the user interface after compile time. You can do your part by restricting dependencies on the generated module, should you later decide to adopt a different tool.

Much work in different user interface separation patterns occurred in the 1980s and 90s. With the advent of the web and the modernization of the model-view-controller (MVC) pattern to reflect web interfaces, MVC has become the dominant separation pattern. Now the MVC pattern is built into a wide variety of different frameworks. (See Chapter 14 for a discussion of MVC.) MVC makes it easy to provide multiple views of the data, supporting user initiative, as we discuss next.

Many times quality attributes are in conflict with each other. Usability and modifiability, on the other hand, often complement each other, because one of the best ways to make a system more usable is to make it modifiable. However, this is not always the case. In many systems business rules drive the UI—for example, specifying how to validate input. To realize this validation, the UI may need to call a server (which can negatively affect performance). To get around this performance penalty, the architect may choose to duplicate these rules in the client and the server, which then makes evolution difficult. Alas, the architect's life is never easy!

There is a connection between the achievement of usability and modifiability. The user interface design process consists of generating and then testing a user interface design. Deficiencies in the design are corrected and the process repeats. If the user interface has already been constructed as a portion of the system, then the system must be modified to reflect the latest design. Hence the connection with modifiability. This connection has resulted in standard patterns to support user interface design (see sidebar).

Support User Initiative

Once a system is executing, usability is enhanced by giving the user feedback as to what the system is doing and by allowing the user to make appropriate responses. For example, the tactics described next—*cancel*, *undo*, *pause/resume*, and *aggregate*—support the user in either correcting errors or being more efficient.

The architect designs a response for user initiative by enumerating and allocating the responsibilities of the system to respond to the user command. Here are some common examples of user initiative:

- *Cancel*. When the user issues a cancel command, the system must be listening for it (thus, there is the responsibility to have a constant listener that is not blocked by the actions of whatever is being canceled); the command being canceled must be terminated; any resources being used by the canceled command must be freed; and components that are collaborating with the canceled command must be informed so that they can also take appropriate action.
- *Undo*. To support the ability to undo, the system must maintain a sufficient amount of information about system state so that an earlier state may be restored, at the user's request. Such a record may be in the form of state "snapshots"—for example, checkpoints—or as a set of reversible operations. Not all operations can be easily reversed: for example, changing all occurrences of the letter "a" to the letter "b" in a document cannot be reversed by changing all instances of "b" to "a", because some of those instances of "b" may have existed prior to the original change. In such a case the system must maintain a more elaborate record of the change. Of course, some operations, such as ringing a bell, cannot be undone.
- *Pause/resume*. When a user has initiated a long-running operation—say, downloading a large file or set of files from a server—it is often useful to provide the ability to pause and resume the operation. Effectively pausing a long-running operation requires the ability to temporarily free resources so that they may be reallocated to other tasks.

- *Aggregate*. When a user is performing repetitive operations, or operations that affect a large number of objects in the same way, it is useful to provide the ability to aggregate the lower-level objects into a single group, so that the operation may be applied to the group, thus freeing the user from the drudgery (and potential for mistakes) of doing the same operation repeatedly. For example, aggregate all of the objects in a slide and change the text to 14-point font.

Support System Initiative

When the system takes the initiative, it must rely on a model of the user, the task being undertaken by the user, or the system state itself. Each model requires various types of input to accomplish its initiative. The *support system initiative* tactics are those that identify the models the system uses to predict either its own behavior or the user's intention. Encapsulating this information will make it easier for it to be tailored or modified. Tailoring and modification can be either dynamically based on past user behavior or offline during development. These tactics are the following:

- *Maintain task model*. The task model is used to determine context so the system can have some idea of what the user is attempting and provide assistance. For example, knowing that sentences start with capital letters would allow an application to correct a lowercase letter in that position.
- *Maintain user model*. This model explicitly represents the user's knowledge of the system, the user's behavior in terms of expected response time, and other aspects specific to a user or a class of users. For example, maintaining a user model allows the system to pace mouse selection so that not all of the document is selected when scrolling is required. Or a model can control the amount of assistance and suggestions automatically provided to a user. A special case of this tactic is commonly found in user interface *customization*, wherein a user can explicitly modify the system's user model.
- *Maintain system model*. Here the system maintains an explicit model of itself. This is used to determine expected system behavior so that appropriate feedback can be given to the user. A common manifestation of a system model is a progress bar that predicts the time needed to complete the current activity.

Figure 11.3 shows a summary of the tactics to achieve usability.

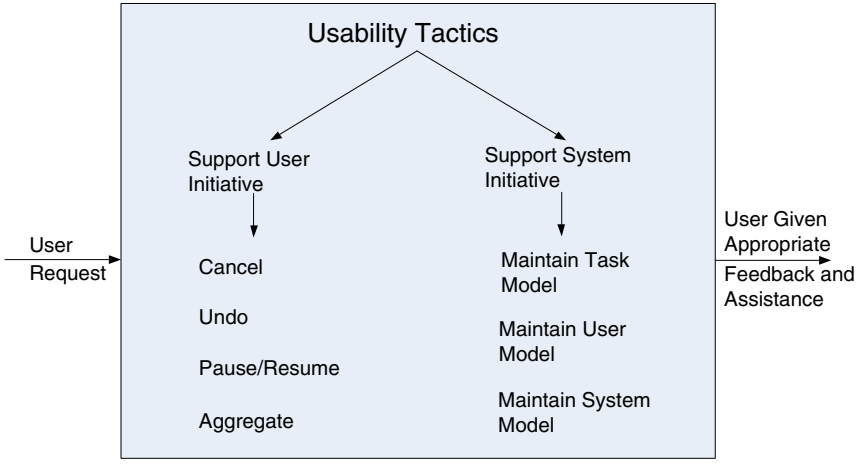

FIGURE 11.3 Usability tactics

11.3 A Design Checklist for Usability

Table 11.2 is a checklist to support the design and analysis process for usability.

TABLE 11.2 Checklist to Support the Design and Analysis Process for Usability

Category	Checklist
Allocation of Responsibilities	Ensure that additional system responsibilities have been allocated, as needed, to assist the user in the following: ▪ Learning how to use the system ▪ Efficiently achieving the task at hand ▪ Adapting and configuring the system ▪ Recovering from user and system errors
Coordination Model	Determine whether the properties of system elements' coordination—timeliness, currency, completeness, correctness, consistency—affect how a user learns to use the system, achieves goals or completes tasks, adapts and configures the system, recovers from user and system errors, and gains increased confidence and satisfaction. For example, can the system respond to mouse events and give semantic feedback in real time? Can long-running events be canceled in a reasonable amount of time?

continues

TABLE 11.2 Checklist to Support the Design and Analysis Process for Usability, *continued*

Category	Checklist
Data Model	Determine the major data abstractions that are involved with user-perceivable behavior. Ensure these major data abstractions, their operations, and their properties have been designed to assist the user in achieving the task at hand, adapting and configuring the system, recovering from user and system errors, learning how to use the system, and increasing satisfaction and user confidence. For example, the data abstractions should be designed to support *undo* and *cancel* operations: the transaction granularity should not be so great that canceling or undoing an operation takes an excessively long time.
Mapping among Architectural Elements	Determine what mapping among architectural elements is visible to the end user (for example, the extent to which the end user is aware of which services are local and which are remote). For those that are visible, determine how this affects the ways in which, or the ease with which, the user will learn how to use the system, achieve the task at hand, adapt and configure the system, recover from user and system errors, and increase confidence and satisfaction.
Resource Management	Determine how the user can adapt and configure the system's use of resources. Ensure that resource limitations under all user-controlled configurations will not make users less likely to achieve their tasks. For example, attempt to avoid configurations that would result in excessively long response times. Ensure that the level of resources will not affect the users' ability to learn how to use the system, or decrease their level of confidence and satisfaction with the system.
Binding Time	Determine which binding time decisions should be under user control and ensure that users can make decisions that aid in usability. For example, if the user can choose, at runtime, the system's configuration, or its communication protocols, or its functionality via plug-ins, you need to ensure that such choices do not adversely affect the user's ability to learn system features, use the system efficiently, minimize the impact of errors, further adapt and configure the system, or increase confidence and satisfaction.
Choice of Technology	Ensure the chosen technologies help to achieve the usability scenarios that apply to your system. For example, do these technologies aid in the creation of online help, the production of training materials, and the collection of user feedback? How usable are any of your chosen technologies? Ensure the chosen technologies do not adversely affect the usability of the system (in terms of learning system features, using the system efficiently, minimizing the impact of errors, adapting/configuring the system, and increasing confidence and satisfaction).

11.4 Summary

Architectural support for usability involves both allowing the user to take the initiative—in circumstances such as canceling a long-running command or undoing a completed command—and aggregating data and commands.

To be able to predict user or system responses, the system must keep an explicit model of the user, the system, and the task.

There is a strong relationship between supporting the user interface design process and supporting modifiability; this relation is promoted by patterns that enforce separation of the user interface from the rest of the system, such as the MVC pattern.

11.5 For Further Reading

Claire Marie Karat has investigated the relation between usability and business advantage [Karat 94].

Jakob Nielsen has also written extensively on this topic, including a calculation on the ROI of usability [Nielsen 08].

Bonnie John and Len Bass have investigated the relation between usability and software architecture. They have enumerated around two dozen usability scenarios that have architectural impact and given associated patterns for these scenarios [Bass 03].

Greg Hartman has defined attentiveness as the ability of the system to support user initiative and allow cancel or pause/resume [Hartman 10].

Some of the patterns for separating the user interface are Arch/Slinky, Seeheim, and PAC. These are discussed in Chapter 8 of *Human-Computer Interaction* [Dix 04].

11.6 Discussion Questions

1. Write a concrete usability scenario for your automobile that specifies how long it takes you to set your favorite radio stations? Now consider another part of the driver experience and create scenarios that test other aspects of the response measures from the general scenario table.
2. Write a concrete usability scenario for an automatic teller machine. How would your design be modified to satisfy these scenarios?

3. How might usability trade off against security? How might it trade off against performance?
4. Pick a few of your favorite web sites that do similar things, such as social networking or online shopping. Now pick one or two appropriate responses from the usability general scenario (such as "achieve the task at hand") and a correspondingly appropriate response measure. Using the response and response measure you chose, compare the web sites' usability.
5. Specify the data model for a four-function calculator that allows undo.
6. Why is it that in so many systems, the cancel button in a dialog box appears to be unresponsive? What architectural principles do you think were ignored in these systems?
7. Why do you think that progress bars frequently behave erratically, moving from 10 to 90 percent in one step and then getting stuck on 90 percent?
8. Research the crash of Air France Flight 296 into the forest at Habsheim, France, on June 26, 1988. The pilots said they were unable to read the digital display of the radio altimeter or hear its audible readout. If they could have, do you believe the crash would have been averted? In this context, discuss the relationship between usability and safety.